

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

Bộ môn: Công nghệ thông tin



BÀI TẬP LỚN

MÔN HỌC

KIỂM THỬ PHẦN MỀM

**ĐỀ TÀI : SPECTRUM-BASED FAULT LOCALIZATION VÀ
SUSPICIOUSNESS SCORE**

GIÁO VIÊN HƯỚNG DẪN : TS. NGUYỄN TUẤN LINH

HỌ VÀ TÊN SINH VIÊN : NGUYỄN ĐÌNH TUẤN HÀ

LỚP : K58KTP

MSSV : K225480106011

THÁI NGUYỄN – 2026

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP

KHOA ĐIỆN TỬ

Bộ môn: Công nghệ thông tin



BÀI TẬP LỚN

MÔN HỌC

KIỂM THỬ PHẦN MỀM

**ĐỀ TÀI : SPECTRUM-BASED FAULT LOCALIZATION VÀ
SUSPICIOUSNESS SCORE**

GIÁO VIÊN HƯỚNG DẪN : TS. NGUYỄN TUẤN LINH

HỌ VÀ TÊN SINH VIÊN : NGUYỄN ĐÌNH TUẤN HÀ

LỚP : K58KTP

MSSV : K225480106011

THÁI NGUYÊN – 2026

BÀI TẬP LỚN

MÔN HỌC: KIỂM THỬ PHẦN MỀM

BỘ MÔN: CÔNG NGHỆ THÔNG TIN

Sinh viên: *Nguy Đình Tuấn Hà*..... Lớp: *K58KTP.k01*

.....

Ngành : *Công Nghệ Phần Mềm*.....

Giáo viên hướng dẫn: *TS. Nguyễn Tuấn Linh*.....

Ngày giao đề: *1/6/2025* Ngày hoàn thành: *25/6/2026*

.....

Tên đề tài:

Coincidentally correct executions trong định vị lỗi tự động

Mô tả bài toán:

Dữ liệu sử dụng trong nghiên cứu gồm:

- Bộ dữ liệu thực thi (Execution Data) thu được từ quá trình chạy chương trình với nhiều bộ dữ liệu kiểm thử khác nhau.
- Các Execution Trace ghi nhận chuỗi câu lệnh được thực thi trong từng lần chạy chương trình.
- Các Execution Path được xây dựng từ Execution Trace nhằm mô tả hành vi thực thi của chương trình.
- Kết quả Pass/Fail tương ứng với từng ca kiểm thử.
- Tập các N-Gram được sinh ra từ Execution Path.
- Giá trị Entropy và Cross Entropy được sử dụng để đánh giá mức độ tương đồng giữa các lần thực thi.

- Danh sách các Coincidentally Correct Executions được phát hiện trong quá trình phân tích.

Bộ dữ liệu kiểm thử:

Test Case	Trạng thái	Mô tả
T1	Pass	Chương trình thực thi đúng
T2	Pass	Chương trình thực thi đúng
T3	Fail	Chương trình cho kết quả sai
T4	Fail	Chương trình cho kết quả sai
T5	Pass	Có khả năng là Coincidentally Correct Execution
T6	Pass	Có khả năng là Coincidentally Correct Execution

Phương pháp thực hiện:

Sử dụng phương pháp phân tích hành vi thực thi của chương trình để phát hiện Coincidentally Correct Executions thông qua:

- Execution Trace
- Execution Path
- Mô hình N-Gram
- Entropy
- Cross Entropy

Quy trình thực hiện:

- Xây dựng chương trình thử nghiệm có chứa lỗi.
- Thiết kế bộ kiểm thử.
- Thu thập Execution Trace từ các lần thực thi.
- Xây dựng Execution Path.
- Sinh tập N-Gram từ Execution Path.
- Tính toán Entropy và Cross Entropy.

- Phát hiện các Coincidentally Correct Executions.
- Đánh giá ảnh hưởng của Coincidentally Correct Executions tới quá trình định vị lỗi tự động.

Công cụ sử dụng:

- Ngôn ngữ lập trình Python.
- Jupyter Notebook.
- Thư viện Python tiêu chuẩn.
- Microsoft Word để xây dựng báo cáo.

Giao diện thực nghiệm:

Bài thực nghiệm được triển khai trên môi trường Jupyter Notebook.

Kết quả hiển thị gồm:

- Danh sách các test case.
- Kết quả Pass/Fail.
- Execution Trace của từng lần thực thi
- Execution Path tương ứng.
- Tập N-Gram được sinh ra.
- Giá trị Entropy và Cross Entropy.
- Danh sách Coincidentally Correct Executions được phát hiện.
- Kết quả đánh giá ảnh hưởng tới Fault Localization.

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, ngày....tháng.....năm 20....

GIÁO VIÊN HƯỚNG DẪN

(Ký ghi rõ họ tên)

MỤC LỤC

LỜI MỞ ĐẦU	8
LỜI CAM ĐOAN	9
LỜI CẢM ƠN.....	10
CHƯƠNG 1. MỞ ĐẦU.....	11
1.1 Lý do chọn đề tài	11
1.2 Mục tiêu nghiên cứu	12
1.3 Phạm vi nghiên cứu	13
1.4 Phương pháp nghiên cứu	14
1.4.1 Phương pháp nghiên cứu tài liệu.....	14
1.4.2 Phương pháp phân tích và tổng hợp.....	14
1.4.3 Phương pháp thực nghiệm	15
1.4.4 Phương pháp đánh giá.....	15
1.5 Ý nghĩa của đề tài	15
1.5.1 Ý nghĩa khoa học	15
1.5.2 Ý nghĩa thực tiễn.....	15
1.6 Cấu trúc của tiểu luận	16
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUÁT	17
2.1 Tổng quan về định vị lỗi tự động.....	17
2.2 Spectrum-Based Fault Localization.....	18
2.3 Coincidentally Correct Executions	19
2.3.1 Khái niệm.....	19
2.3.2 Mô hình RIP.....	20
2.3.3 Weak Coincidental Correctness	20
2.3.4 Strong Coincidental Correctness.....	21

2.3.5 Nguyên nhân xuất hiện Coincidentally Correct Executions	21
2.3.6 Tác động của Coincidentally Correct Executions	22
2.4 Phát hiện Coincidentally Correct Executions	22
2.5 Execution Trace và Execution Path	23
2.6 Mô hình N-Gram	23
2.7 Entropy và Cross Entropy	24
2.8 Liên hệ với chất lượng phần mềm	24
CHƯƠNG 3. VẬN DỤNG VÀ THỰC HÀNH PHÁT HIỆN COINCIDENTALLY CORRECT EXECUTIONS.....	26
3.1 Quy trình phát hiện Coincidentally Correct Executions.....	26
3.2 Xây dựng chương trình thử nghiệm.....	27
3.3 Thiết kế bộ dữ liệu kiểm thử.....	28
3.4 Thu thập Execution Trace và Execution Path.....	30
3.4.1. Cài đặt chương trình thử nghiệm bằng Python	31
3.4.2. Kết quả thu được	37
3.5 Biểu diễn Execution Path bằng mô hình N-Gram	38
3.6 Áp dụng Cross Entropy để phát hiện Coincidentally Correct Executions	38
3.7 Phát hiện Coincidentally Correct Executions	40
3.8 Đánh giá ảnh hưởng tới Fault Localization	40
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	42
4.1 Kết luận.....	42
4.2 Hạn chế của đề tài.....	43
4.3 Hướng phát triển trong tương lai	44
TÀI LIỆU THAM KHẢO	46
PHỤ LỤC	47

LỜI MỞ ĐẦU

Trong quá trình phát triển phần mềm, việc phát hiện và sửa lỗi là một nhiệm vụ quan trọng nhằm đảm bảo chất lượng sản phẩm. Tuy nhiên, đối với các hệ thống có quy mô lớn, việc xác định chính xác vị trí gây lỗi trong mã nguồn thường tốn nhiều thời gian và công sức. Vì vậy, các kỹ thuật định vị lỗi tự động (Automatic Fault Localization) đã được nghiên cứu nhằm hỗ trợ lập trình viên trong quá trình gỡ lỗi.

Một trong những vấn đề ảnh hưởng đến hiệu quả của các kỹ thuật định vị lỗi là hiện tượng Coincidentally Correct Executions (CCE), tức là những trường hợp chương trình thực thi qua đoạn mã chứa lỗi nhưng vẫn tạo ra kết quả đúng. Hiện tượng này có thể làm sai lệch dữ liệu thực thi và làm giảm độ chính xác của quá trình định vị lỗi.

Xuất phát từ đó, em lựa chọn đề tài “Coincidentally Correct Executions trong định vị lỗi tự động” để nghiên cứu. Nội dung tiểu luận tập trung tìm hiểu cơ sở lý thuyết của Coincidentally Correct Executions, các phương pháp phát hiện hiện tượng này và đánh giá ảnh hưởng của chúng đối với quá trình định vị lỗi tự động thông qua một bài toán thực nghiệm.

Do kiến thức và kinh nghiệm còn hạn chế, bài tiểu luận khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy để bài làm được hoàn thiện hơn.

Em xin chân thành cảm ơn.

LỜI CAM ĐOAN

Em xin cam đoan rằng bài tiểu luận với đề tài “Coincidentally Correct Executions trong định vị lỗi tự động” là kết quả nghiên cứu và thực hiện của cá nhân em dưới sự hướng dẫn của giảng viên phụ trách học phần.

Các nội dung, số liệu, hình ảnh và tài liệu tham khảo được sử dụng trong bài tiểu luận đều được thu thập từ các nguồn tài liệu có liên quan và được trích dẫn rõ ràng theo đúng quy định. Những nội dung phân tích, tổng hợp, đánh giá và nhận xét được trình bày trong bài là kết quả nghiên cứu của cá nhân em dựa trên cơ sở tìm hiểu các tài liệu tham khảo về lĩnh vực kiểm thử phần mềm, định vị lỗi tự động và Coincidentally Correct Executions.

Em xin hoàn toàn chịu trách nhiệm về tính trung thực, tính chính xác và tính khách quan của các nội dung được trình bày trong bài tiểu luận này.

Thái Nguyên, ngày tháng năm 2026

Sinh viên thực hiện

Hà

Nguyễn Đình Tuấn Hà

LỜI CẢM ƠN

Trong quá trình thực hiện và hoàn thành bài tiểu luận môn Kiểm thử phần mềm, em đã nhận được sự giúp đỡ, hướng dẫn và tạo điều kiện từ nhiều cá nhân và tập thể.

Trước hết, em xin gửi lời cảm ơn chân thành và sâu sắc tới TS. Nguyễn Tuấn Linh, giảng viên phụ trách học phần Kiểm thử phần mềm, người đã tận tình hướng dẫn, truyền đạt những kiến thức chuyên môn quý báu và tạo điều kiện thuận lợi để em hoàn thành bài tiểu luận này.

Em cũng xin cảm ơn các thầy cô trong Bộ môn Công nghệ Thông tin đã trang bị cho em những kiến thức nền tảng trong suốt quá trình học tập, giúp em có đủ kiến thức để thực hiện đề tài nghiên cứu.

Bên cạnh đó, em xin cảm ơn gia đình, bạn bè và các bạn cùng lớp đã luôn động viên, hỗ trợ và chia sẻ kinh nghiệm trong quá trình học tập cũng như hoàn thành bài tiểu luận.

Mặc dù đã cố gắng hoàn thành bài tiểu luận với tinh thần nghiêm túc và trách nhiệm cao nhất, tuy nhiên do kiến thức và thời gian thực hiện còn hạn chế nên bài làm khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp quý báu từ thầy để bài tiểu luận được hoàn thiện hơn.

Em xin chân thành cảm ơn!

CHƯƠNG 1. MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong những năm gần đây, quy mô và độ phức tạp của các hệ thống phần mềm ngày càng gia tăng. Các ứng dụng hiện đại không chỉ bao gồm hàng nghìn hoặc hàng triệu dòng mã nguồn mà còn phải đáp ứng các yêu cầu khắt khe về hiệu năng, độ tin cậy và khả năng bảo trì. Trong bối cảnh đó, việc phát hiện và xử lý lỗi phần mềm trở thành một nhiệm vụ quan trọng nhằm đảm bảo chất lượng sản phẩm cũng như giảm thiểu các rủi ro trong quá trình vận hành hệ thống.

Mặc dù các kỹ thuật kiểm thử phần mềm ngày càng phát triển và có khả năng phát hiện lỗi hiệu quả hơn, việc xác định chính xác vị trí gây ra lỗi trong mã nguồn vẫn là một thách thức lớn đối với các nhà phát triển phần mềm. Trên thực tế, sau khi một lỗi được phát hiện thông qua quá trình kiểm thử, lập trình viên thường phải dành nhiều thời gian để phân tích chương trình, theo dõi luồng thực thi và tìm kiếm nguyên nhân gốc rễ của vấn đề. Quá trình này được gọi là định vị lỗi (Fault Localization).

giảm thiểu công sức và thời gian dành cho hoạt động định vị lỗi, nhiều phương pháp định vị lỗi tự động đã được đề xuất. Một trong những hướng tiếp cận phổ biến nhất hiện nay là Spectrum-Based Fault Localization (SBFL). Phương pháp này khai thác dữ liệu thực thi của các ca kiểm thử thành công và thất bại để xác định mức độ nghi ngờ của từng thành phần trong chương trình. Nhờ tính đơn giản và hiệu quả, SBFL đã trở thành nền tảng cho nhiều nghiên cứu trong lĩnh vực tự động hóa kiểm thử phần mềm.

Tuy nhiên, hiệu quả của các kỹ thuật định vị lỗi dựa trên phổ thực thi phụ thuộc rất lớn vào chất lượng của dữ liệu kiểm thử. Trong quá trình thực thi chương trình có thể xuất hiện những trường hợp đặc biệt mà một ca kiểm thử đi qua vùng mã chứa lỗi, thậm chí lỗi đã được kích hoạt, nhưng kết quả cuối cùng vẫn đúng như mong đợi. Những trường hợp này được gọi là Coincidentally Correct

Executions. Sự tồn tại của chúng làm sai lệch các thông tin thống kê được sử dụng trong quá trình định vị lỗi, từ đó làm giảm đáng kể độ chính xác của các thuật toán Fault Localization.

Coincidentally Correct Executions hiện được xem là một trong những nguyên nhân quan trọng ảnh hưởng đến hiệu quả của nhiều phương pháp định vị lỗi tự động. Việc nghiên cứu bản chất của hiện tượng này, tìm hiểu nguyên nhân hình thành cũng như các phương pháp phát hiện và xử lý Coincidentally Correct Executions có ý nghĩa lớn trong việc nâng cao hiệu quả của các hệ thống kiểm thử phần mềm hiện đại.

Xuất phát từ những lý do trên, đề tài “Coincidentally Correct Executions trong định vị lỗi tự động” được lựa chọn nhằm nghiên cứu sâu hơn về hiện tượng Coincidentally Correct Executions, đánh giá tác động của nó đối với quá trình định vị lỗi và tìm hiểu các giải pháp giúp cải thiện độ chính xác của các kỹ thuật Fault Localization.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu hiện tượng Coincidentally Correct Executions trong quá trình định vị lỗi tự động, từ đó đánh giá ảnh hưởng của hiện tượng này đến hiệu quả của các phương pháp Fault Localization và đề xuất hướng tiếp cận nhằm giảm thiểu các tác động tiêu cực của nó.

Để đạt được mục tiêu tổng quát nêu trên, đề tài tập trung thực hiện các mục tiêu cụ thể sau:

- Nghiên cứu các khái niệm cơ bản liên quan đến lỗi phần mềm, kiểm thử phần mềm và định vị lỗi tự động.
- Tìm hiểu nguyên lý hoạt động của các phương pháp Spectrum-Based Fault Localization.
- Phân tích bản chất của hiện tượng Coincidentally Correct Executions.

- Làm rõ các điều kiện dẫn đến sự xuất hiện của Coincidentally Correct Executions trong quá trình thực thi chương trình.
- Phân tích ảnh hưởng của Coincidentally Correct Executions đối với độ chính xác của các thuật toán định vị lỗi.
- Nghiên cứu các phương pháp phát hiện Coincidentally Correct Executions dựa trên dữ liệu thực thi chương trình.
- Tìm hiểu việc ứng dụng mô hình N-Gram và độ đo Cross Entropy trong phát hiện Coincidentally Correct Executions.
- Xây dựng ví dụ thực nghiệm để minh họa quá trình phát hiện Coincidentally Correct Executions.
- Đánh giá hiệu quả của phương pháp được áp dụng trong việc hỗ trợ định vị lỗi tự động.

1.3 Phạm vi nghiên cứu

Do giới hạn về thời gian thực hiện và phạm vi của môn học, đề tài tập trung nghiên cứu các nội dung liên quan trực tiếp đến Coincidentally Correct Executions và các phương pháp phát hiện hiện tượng này trong bối cảnh định vị lỗi tự động.

Cụ thể, phạm vi nghiên cứu của đề tài bao gồm:

- Các khái niệm cơ bản về lỗi phần mềm và quá trình định vị lỗi.
- Các phương pháp định vị lỗi tự động dựa trên phổ thực thi.
- Hiện tượng Coincidentally Correct Executions.
- Phân tích dữ liệu thực thi chương trình.
- Execution Trace và Execution Path.
- Mô hình N-Gram trong phân tích hành vi thực thi.
- Độ đo Entropy và Cross Entropy.

- Phương pháp phát hiện Coincidentally Correct Executions dựa trên đặc trưng thực thi.

Đề tài không đi sâu nghiên cứu các lĩnh vực như:

- Trí tuệ nhân tạo trong sửa lỗi tự động.
- Deep Learning Fault Localization.
- Mutation Testing.
- Search-Based Software Engineering.
- Các kỹ thuật sửa lỗi chương trình tự động.

Việc giới hạn phạm vi nghiên cứu giúp tập trung làm rõ những nội dung cốt lõi liên quan đến Coincidentally Correct Executions và đảm bảo chiều sâu phân tích của đề tài.

1.4 Phương pháp nghiên cứu

Để thực hiện các mục tiêu đã đề ra, đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu khác nhau.

1.4.1 Phương pháp nghiên cứu tài liệu

Đây là phương pháp được sử dụng xuyên suốt quá trình thực hiện đề tài. Các tài liệu chuyên ngành về kiểm thử phần mềm, định vị lỗi tự động và Coincidentally Correct Executions được thu thập, phân tích và tổng hợp nhằm xây dựng cơ sở lý thuyết cho nghiên cứu.

Bên cạnh tài liệu chính được chỉ định, đề tài còn tham khảo thêm các bài báo khoa học, sách chuyên khảo và các công trình nghiên cứu liên quan nhằm mở rộng góc nhìn và tăng tính khách quan của nội dung nghiên cứu.

1.4.2 Phương pháp phân tích và tổng hợp

Sau khi thu thập tài liệu, các khái niệm, mô hình và phương pháp nghiên cứu được phân tích nhằm làm rõ bản chất của vấn đề. Những thông tin thu được

được tổng hợp và hệ thống hóa thành các nội dung lý thuyết phục vụ cho quá trình nghiên cứu.

1.4.3 Phương pháp thực nghiệm

Đề tài xây dựng một chương trình minh họa và thiết kế các trường hợp kiểm thử nhằm mô phỏng quá trình thực thi chương trình có chứa Coincidentally Correct Executions. Thông qua dữ liệu thực thi thu được, các phương pháp phát hiện Coincidentally Correct Executions được áp dụng để đánh giá tính khả thi và hiệu quả của giải pháp.

1.4.4 Phương pháp đánh giá

Kết quả thực nghiệm được phân tích và so sánh nhằm đánh giá khả năng phát hiện Coincidentally Correct Executions cũng như mức độ cải thiện độ chính xác của quá trình định vị lỗi sau khi xử lý các trường hợp này.

1.5 Ý nghĩa của đề tài

1.5.1 Ý nghĩa khoa học

Đề tài góp phần làm rõ bản chất của hiện tượng Coincidentally Correct Executions trong lĩnh vực định vị lỗi tự động. Thông qua việc nghiên cứu các cơ chế hình thành và phương pháp phát hiện Coincidentally Correct Executions, đề tài giúp bổ sung thêm cơ sở lý luận cho các nghiên cứu liên quan đến Fault Localization.

Ngoài ra, việc phân tích mối quan hệ giữa Coincidentally Correct Executions và Spectrum-Based Fault Localization còn giúp làm rõ những hạn chế của các phương pháp định vị lỗi hiện nay, từ đó tạo tiền đề cho các hướng nghiên cứu cải tiến trong tương lai.

1.5.2 Ý nghĩa thực tiễn

Trong thực tế phát triển phần mềm, việc xác định nhanh và chính xác vị trí lỗi có ý nghĩa rất lớn đối với hiệu quả của quá trình bảo trì hệ thống. Nếu các

trường hợp Coincidentally Correct Executions không được xử lý phù hợp, kết quả định vị lỗi có thể bị sai lệch, làm gia tăng thời gian và chi phí gỡ lỗi.

Nghiên cứu hiện tượng Coincidentally Correct Executions và các phương pháp phát hiện chúng giúp nâng cao chất lượng dữ liệu phục vụ định vị lỗi, từ đó hỗ trợ lập trình viên xác định lỗi nhanh hơn và hiệu quả hơn. Điều này góp phần giảm chi phí phát triển phần mềm, nâng cao chất lượng sản phẩm và tăng độ tin cậy của hệ thống.

1.6 Cấu trúc của tiểu luận

Ngoài phần mở đầu, kết luận và tài liệu tham khảo, nội dung chính của tiểu luận được tổ chức thành ba chương.

Chương 2 trình bày cơ sở lý thuyết và tổng quan về định vị lỗi tự động, các kỹ thuật Spectrum-Based Fault Localization cũng như hiện tượng Coincidentally Correct Executions và các phương pháp phát hiện hiện tượng này.

Chương 3 tập trung xây dựng bài toán thực nghiệm, mô tả quy trình áp dụng phương pháp phát hiện Coincidentally Correct Executions dựa trên dữ liệu thực thi chương trình, đồng thời đánh giá kết quả đạt được.

Chương 4 trình bày các kết luận chính của đề tài, những hạn chế còn tồn tại và các hướng nghiên cứu có thể tiếp tục phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUÁT

2.1 Tổng quan về định vị lỗi tự động

Trong quá trình phát triển phần mềm, lỗi là hiện tượng khó có thể tránh khỏi. Mặc dù các kỹ thuật kiểm thử hiện đại có khả năng phát hiện sự tồn tại của lỗi, việc xác định chính xác vị trí gây ra lỗi trong mã nguồn vẫn là một nhiệm vụ phức tạp và tốn nhiều thời gian. Đối với các hệ thống phần mềm có quy mô lớn, số lượng dòng mã nguồn có thể lên tới hàng trăm nghìn hoặc hàng triệu dòng, khiến việc tìm kiếm thủ công vị trí lỗi trở nên khó khăn và kém hiệu quả.

Định vị lỗi (Fault Localization) là quá trình xác định những thành phần của chương trình có khả năng chứa lỗi nhằm hỗ trợ lập trình viên trong quá trình gỡ lỗi. Mục tiêu của định vị lỗi không phải là sửa lỗi một cách tự động mà là thu hẹp phạm vi tìm kiếm, từ đó giúp người phát triển tập trung vào những khu vực có khả năng gây ra lỗi cao nhất.

Sự phát triển của các hệ thống phần mềm hiện đại đã thúc đẩy nhu cầu nghiên cứu các phương pháp định vị lỗi tự động. Thay vì dựa hoàn toàn vào kinh nghiệm của lập trình viên, các kỹ thuật định vị lỗi tự động sử dụng dữ liệu thực thi chương trình, thông tin kiểm thử hoặc các mô hình thống kê để xác định vị trí nghi ngờ. Nhờ đó, thời gian gỡ lỗi được rút ngắn đáng kể và hiệu quả bảo trì phần mềm được nâng cao.

Hiện nay có nhiều hướng tiếp cận khác nhau trong định vị lỗi tự động như Program Slicing, Model-Based Fault Localization, Machine Learning-Based Fault Localization và Spectrum-Based Fault Localization. Trong số đó, Spectrum-Based Fault Localization được xem là một trong những phương pháp phổ biến nhất nhờ tính đơn giản, hiệu quả và khả năng áp dụng trên nhiều loại chương trình khác nhau.

2.2 Spectrum-Based Fault Localization

Spectrum-Based Fault Localization (SBFL) là phương pháp định vị lỗi dựa trên dữ liệu thực thi của chương trình. Ý tưởng cốt lõi của SBFL là sử dụng thông tin về các câu lệnh đã được thực thi trong các ca kiểm thử thành công và thất bại để xác định mức độ nghi ngờ của từng thành phần trong chương trình.

Trong quá trình kiểm thử, mỗi bộ dữ liệu đầu vào tạo ra một lần thực thi riêng biệt. Kết quả thực thi có thể được phân loại thành hai nhóm:

- Passing Execution: thực thi thành công.
- Failing Execution: thực thi thất bại.

Đối với mỗi lần thực thi, hệ thống ghi nhận những câu lệnh đã được chạy. Tập hợp các thông tin này được sử dụng để xây dựng ma trận bao phủ (Coverage Matrix), thể hiện mối quan hệ giữa các ca kiểm thử và các câu lệnh trong chương trình.

Nguyên lý hoạt động của SBFL dựa trên giả định rằng các câu lệnh xuất hiện thường xuyên trong các lần thực thi thất bại có khả năng chứa lỗi cao hơn các câu lệnh chỉ xuất hiện trong các lần thực thi thành công. Từ dữ liệu bao phủ, nhiều công thức thống kê đã được đề xuất nhằm tính toán độ nghi ngờ (Suspiciousness Score) cho từng câu lệnh.

Một số công thức phổ biến bao gồm:

Tarantula

Tarantula là một trong những công thức đầu tiên được sử dụng trong SBFL. Công thức này đánh giá mức độ nghi ngờ của một câu lệnh dựa trên tỷ lệ xuất hiện của câu lệnh trong các lần thực thi thất bại và thành công.

Ochiai

Ochiai được phát triển dựa trên hệ số tương đồng trong khai phá dữ liệu và hiện được xem là một trong những công thức hiệu quả nhất trong lĩnh vực định vị lỗi tự động.

Jaccard

Jaccard đánh giá mức độ tương quan giữa việc thực thi câu lệnh và kết quả thất bại của chương trình thông qua hệ số tương đồng Jaccard.

DStar

DStar là một công thức cải tiến nhằm tăng cường khả năng phân biệt giữa các câu lệnh lỗi và các câu lệnh không lỗi bằng cách nhấn mạnh vai trò của các lần thực thi thất bại.

Mặc dù đạt được nhiều kết quả tích cực, hiệu quả của SBFL phụ thuộc rất lớn vào chất lượng dữ liệu thực thi. Một trong những yếu tố có thể làm sai lệch dữ liệu thống kê là sự xuất hiện của Coincidentally Correct Executions.

2.3 Coincidentally Correct Executions

2.3.1 Khái niệm

Coincidentally Correct Executions (CCE) là những lần thực thi mà chương trình tạo ra kết quả đầu ra đúng mặc dù đã đi qua vùng mã chứa lỗi. Trong các trường hợp này, lỗi đã được thực thi nhưng không dẫn đến thất bại có thể quan sát được ở kết quả cuối cùng.

Thông thường, người ta có xu hướng cho rằng nếu chương trình đi qua một câu lệnh lỗi thì kết quả sẽ sai. Tuy nhiên trên thực tế điều này không phải lúc nào cũng đúng. Có nhiều tình huống mà lỗi đã được kích hoạt nhưng ảnh hưởng của nó bị triệt tiêu hoặc không lan truyền tới đầu ra cuối cùng. Kết quả là chương trình vẫn trả về giá trị đúng và ca kiểm thử được đánh dấu là thành công.

Sự tồn tại của Coincidentally Correct Executions là nguyên nhân gây khó khăn cho các phương pháp định vị lỗi dựa trên thống kê vì chúng làm sai lệch mối quan hệ giữa hành vi thực thi và kết quả kiểm thử.

2.3.2 Mô hình RIP

Để hiểu rõ nguyên nhân xuất hiện Coincidentally Correct Executions, cần xem xét mô hình RIP (Reachability – Infection – Propagation).

Theo mô hình này, để một lỗi dẫn đến thất bại có thể quan sát được cần thỏa mãn đồng thời ba điều kiện:

- Reachability là điều kiện đầu tiên. Câu lệnh chứa lỗi phải được thực thi trong quá trình chạy chương trình. Nếu lỗi không được thực thi thì sẽ không thể ảnh hưởng đến kết quả.
- Infection là điều kiện thứ hai. Sau khi thực thi lỗi, chương trình phải xuất hiện trạng thái sai. Trạng thái sai có thể là giá trị biến không chính xác hoặc dữ liệu nội bộ không đúng với mong đợi.
- Propagation là điều kiện cuối cùng. Trạng thái sai phải lan truyền tới đầu ra của chương trình và làm thay đổi kết quả cuối cùng.

Nếu một trong ba điều kiện trên không được thỏa mãn thì lỗi có thể tồn tại nhưng không dẫn đến thất bại. Đây chính là cơ sở hình thành Coincidentally Correct Executions.

2.3.3 Weak Coincidental Correctness

Weak Coincidental Correctness xảy ra khi câu lệnh lỗi được thực thi nhưng không tạo ra trạng thái sai trong chương trình.

Trong trường hợp này điều kiện Reachability được thỏa mãn nhưng điều kiện Infection không xảy ra. Mặc dù chương trình đã đi qua vùng mã lỗi, trạng thái nội bộ của hệ thống vẫn không bị ảnh hưởng. Do đó kết quả đầu ra vẫn chính xác.

Loại Coincidental Correctness này thường xuất hiện khi dữ liệu đầu vào không kích hoạt điều kiện gây lỗi hoặc khi lỗi nằm trong một nhánh chương trình không ảnh hưởng tới phép tính hiện tại.

2.3.4 Strong Coincidental Correctness

Strong Coincidental Correctness xảy ra khi lỗi đã được thực thi và trạng thái sai đã xuất hiện nhưng trạng thái sai không lan truyền tới đầu ra cuối cùng.

Trong trường hợp này cả Reachability và Infection đều được thỏa mãn, tuy nhiên điều kiện Propagation không xảy ra.

Ví dụ: Một biến có thể nhận giá trị sai tại một thời điểm nào đó nhưng sau đó được cập nhật lại bằng một giá trị đúng trước khi chương trình kết thúc. Mặc dù trạng thái sai từng tồn tại, người sử dụng vẫn quan sát được kết quả chính xác.

Strong Coincidental Correctness thường gây khó khăn hơn trong quá trình phát hiện vì hành vi thực thi của nó rất giống với các lần thực thi thất bại.

2.3.5 Nguyên nhân xuất hiện Coincidentally Correct Executions

Coincidentally Correct Executions có thể xuất hiện do nhiều nguyên nhân khác nhau.

Thứ nhất: Dữ liệu đầu vào không đủ để kích hoạt toàn bộ ảnh hưởng của lỗi. Mặc dù chương trình đi qua vùng mã lỗi nhưng các điều kiện cần thiết để tạo ra trạng thái sai không xảy ra.

Thứ hai: Trạng thái sai bị ghi đè bởi các phép tính tiếp theo. Giá trị không chính xác được sinh ra nhưng sau đó được thay thế bằng một giá trị đúng trước khi ảnh hưởng tới đầu ra.

Thứ ba: Lỗi chỉ ảnh hưởng tới một phần dữ liệu nhưng phần dữ liệu đó không được sử dụng trong quá trình tạo kết quả cuối cùng.

Thứ tư: Các phép tính bù trừ trong chương trình vô tình loại bỏ tác động của lỗi và dẫn đến kết quả chính xác.

Những nguyên nhân trên cho thấy việc thực thi lỗi không đồng nghĩa với việc chương trình chắc chắn thất bại.

2.3.6 Tác động của Coincidentally Correct Executions

Coincidentally Correct Executions có ảnh hưởng đáng kể đến hiệu quả của các kỹ thuật định vị lỗi dựa trên thống kê.

Trong SBFL, độ nghi ngờ của một câu lệnh được tính dựa trên số lần câu lệnh đó xuất hiện trong các lần thực thi thành công và thất bại. Khi một Coincidentally Correct Execution bị xem như một lần thực thi thành công thông thường, câu lệnh lỗi sẽ bị ghi nhận vào nhóm Passing Execution.

Điều này làm giảm độ nghi ngờ của câu lệnh lỗi và khiến kết quả xếp hạng trở nên kém chính xác hơn. Trong nhiều trường hợp, câu lệnh chứa lỗi có thể bị xếp sau nhiều câu lệnh hoàn toàn không liên quan đến lỗi.

Do đó, việc nhận diện và xử lý Coincidentally Correct Executions là một bước quan trọng nhằm nâng cao hiệu quả của các phương pháp Fault Localization.

2.4 Phát hiện Coincidentally Correct Executions

Việc phát hiện Coincidentally Correct Executions là một thách thức bởi kết quả đầu ra của các lần thực thi này hoàn toàn chính xác. Nếu chỉ dựa vào kết quả Pass hoặc Fail thì rất khó phân biệt chúng với các lần thực thi thành công thông thường.

Một hướng tiếp cận phổ biến là phân tích hành vi thực thi của chương trình. Ý tưởng cơ bản là nếu một lần thực thi thành công có đặc điểm rất giống với các lần thực thi thất bại thì khả năng cao đó là một Coincidentally Correct Execution.

Để thực hiện điều này, cần thu thập và phân tích thông tin về đường thực thi của chương trình thông qua execution trace và execution path.

2.5 Execution Trace và Execution Path

Execution Trace là chuỗi các câu lệnh hoặc sự kiện được ghi nhận trong quá trình thực thi chương trình. Mỗi bộ dữ liệu kiểm thử sẽ tạo ra một execution trace riêng phản ánh hành vi thực tế của chương trình.

Từ execution trace có thể xây dựng execution path, biểu diễn con đường mà chương trình đã đi qua trong quá trình chạy. Execution path thường được mô tả dưới dạng chuỗi các quyết định rẽ nhánh hoặc các predicate xuất hiện trong chương trình.

Việc phân tích execution path cho phép so sánh mức độ tương đồng giữa các lần thực thi khác nhau. Đây là cơ sở quan trọng để nhận diện các trường hợp Coincidentally Correct Executions.

2.6 Mô hình N-Gram

Một phương pháp phổ biến để biểu diễn execution path là sử dụng mô hình N-Gram.

N-Gram là chuỗi liên tiếp gồm n phần tử xuất hiện trong một dãy dữ liệu. Khi áp dụng vào bài toán phân tích thực thi chương trình, mỗi phần tử thường tương ứng với một predicate hoặc một quyết định rẽ nhánh.

Mô hình N-Gram cho phép trích xuất các mẫu hành vi cục bộ xuất hiện trong execution path. Thông qua việc thống kê tần suất xuất hiện của các N-Gram, có thể xây dựng đặc trưng cho từng lần thực thi và so sánh chúng với nhau.

Nhờ khả năng biểu diễn đơn giản nhưng hiệu quả, N-Gram được sử dụng rộng rãi trong nhiều nghiên cứu liên quan đến phát hiện Coincidentally Correct Executions.

2.7 Entropy và Cross Entropy

Entropy là đại lượng dùng để đo mức độ không chắc chắn của một phân bố xác suất. Trong phân tích hành vi thực thi chương trình, Entropy được sử dụng để đánh giá mức độ phân tán của các mẫu N-Gram.

Cross Entropy là độ đo dùng để đánh giá sự khác biệt giữa hai phân bố xác suất. Trong bài toán phát hiện Coincidentally Correct Executions, một phân bố thường được xây dựng từ tập các execution path thất bại, trong khi phân bố còn lại được xây dựng từ execution path đang được xem xét.

Nếu giá trị Cross Entropy nhỏ, điều đó cho thấy hai execution path có mức độ tương đồng cao. Khi một lần thực thi được đánh dấu là thành công nhưng có độ tương đồng lớn với các lần thực thi thất bại, nó có thể được xem là một ứng viên Coincidentally Correct Execution.

Nhờ khả năng đo lường mức độ tương đồng giữa các hành vi thực thi, Cross Entropy trở thành một công cụ hữu ích trong việc phát hiện các trường hợp Coincidentally Correct Executions.

2.8 Liên hệ với chất lượng phần mềm

Việc phát hiện và xử lý Coincidentally Correct Executions mang lại nhiều lợi ích đối với chất lượng phần mềm.

Trước hết, dữ liệu thực thi được làm sạch trước khi áp dụng các kỹ thuật định vị lỗi. Điều này giúp nâng cao độ chính xác của các thuật toán Fault Localization và cải thiện khả năng xác định vị trí lỗi.

Bên cạnh đó, việc giảm thiểu ảnh hưởng của Coincidentally Correct Executions giúp rút ngắn thời gian gỡ lỗi, giảm chi phí bảo trì và nâng cao hiệu quả phát triển phần mềm.

Ngoài ra, khả năng nhận diện chính xác các trường hợp Coincidentally Correct Executions còn góp phần nâng cao độ tin cậy của các hệ thống kiểm thử tự động và hỗ trợ xây dựng các công cụ phân tích phần mềm thông minh trong tương lai.

Như vậy, Coincidentally Correct Executions không chỉ là một hiện tượng lý thuyết trong lĩnh vực định vị lỗi mà còn là một vấn đề có ý nghĩa thực tiễn quan trọng đối với việc đảm bảo chất lượng phần mềm.

CHƯƠNG 3. VẬN DỤNG VÀ THỰC HÀNH PHÁT HIỆN COINCIDENTALLY CORRECT EXECUTIONS

3.1 Quy trình phát hiện Coincidentally Correct Executions

Như đã trình bày trong Chương 2, Coincidentally Correct Executions (CCE) là những lần thực thi mà chương trình tạo ra kết quả đầu ra đúng mặc dù đã đi qua vùng mã chứa lỗi. Đây là một trong những nguyên nhân làm giảm hiệu quả của các kỹ thuật định vị lỗi dựa trên thống kê bởi các trường hợp này thường bị ghi nhận nhầm là các lần thực thi thành công thông thường.

Khác với các trường hợp thất bại, Coincidentally Correct Executions không thể được nhận diện chỉ dựa trên kết quả đầu ra của chương trình. Nếu chỉ quan sát trạng thái Pass hoặc Fail, các trường hợp CCE hoàn toàn không có sự khác biệt với những lần thực thi thành công thực sự. Chính vì vậy cần phải phân tích sâu hơn vào hành vi thực thi bên trong chương trình.

Quá trình phát hiện Coincidentally Correct Executions thường bao gồm các bước cơ bản sau:

- Thu thập dữ liệu thực thi của chương trình.
- Xây dựng execution trace cho từng ca kiểm thử.
- Chuyển đổi execution trace thành execution path.
- Biểu diễn execution path dưới dạng các đặc trưng N-Gram.
- Xây dựng phân bố xác suất của các execution path thất bại.
- Tính toán độ tương đồng giữa các execution path bằng Cross Entropy.
- Xác định các trường hợp Pass có hành vi gần giống với nhóm Fail.

Ý tưởng cốt lõi của phương pháp này là các Coincidentally Correct Executions thường có hành vi thực thi tương đồng với các lần thực thi thất bại. Mặc dù kết

quả cuối cùng là đúng nhưng đường thực thi của chúng vẫn đi qua vùng mã chứa lỗi và mang nhiều đặc điểm giống với các lần thực thi lỗi.

Do đó, nếu có thể đo lường được mức độ tương đồng giữa các execution path thì hoàn toàn có thể nhận diện được các trường hợp Coincidentally Correct Executions.

3.2 Xây dựng chương trình thử nghiệm

Để minh họa quá trình phát hiện Coincidentally Correct Executions, đề tài xây dựng một chương trình xét học bổng sinh viên dựa trên các tiêu chí học tập và rèn luyện.

Theo quy định giả định của nhà trường, sinh viên chỉ được xét học bổng khi:

- Điểm rèn luyện từ 80 trở lên.
- Số tín chỉ tích lũy từ 15 trở lên.

Sau khi đáp ứng hai điều kiện trên, mức học bổng được xác định dựa trên GPA như sau:

GPA	Loại học bổng
< 3.0	Không học bổng
$3.0 - < 3.5$	Khuyến khích
$3.5 - < 3.8$	Giỏi
≥ 3.8	Xuất sắc

Mã giả của chương trình được xây dựng như sau:

```
def scholarship(gpa, conduct, credits):  
  
    if conduct < 80 or credits < 15:  
        return "No Scholarship"
```

```
if gpa >= 3.4:
    return "Excellent"

elif gpa >= 3.5:
    return "Good"

elif gpa >= 3.0:
    return "Encouragement"

return "No Scholarship"
```

Quan sát chương trình có thể thấy tồn tại một lỗi logic.

Điều kiện xác định học bổng Xuất sắc được thiết lập là: $\text{if gpa} \geq 3.4$ trong khi yêu cầu đúng phải là: $\text{if gpa} \geq 3.8$. Lỗi này khiến một số sinh viên được phân loại sai loại học bổng.

Mặc dù đây chỉ là một sai sót nhỏ trong điều kiện rẽ nhánh, nó đủ để tạo ra các trường hợp thực thi thất bại cũng như các Coincidentally Correct Executions phục vụ cho quá trình nghiên cứu.

3.3 Thiết kế bộ dữ liệu kiểm thử

Sau khi xây dựng chương trình, bước tiếp theo là thiết kế bộ dữ liệu kiểm thử.

Mục tiêu của bộ kiểm thử là bao phủ nhiều nhánh thực thi khác nhau của chương trình nhằm tạo điều kiện quan sát rõ ảnh hưởng của lỗi.

Bảng 3.1 trình bày tập dữ liệu kiểm thử được sử dụng.

Test	GPA	Conduct	Credits
T1	2.5	85	18
T2	2.8	90	20
T3	3.0	82	15
T4	3.2	90	18
T5	3.4	88	20
T6	3.5	85	16
T7	3.6	92	22
T8	3.7	95	24
T9	3.8	90	18
T10	3.9	96	25
T11	3.1	75	20
T12	3.9	70	18

Sau khi thực thi chương trình, kết quả thu được như sau.

Test	Kết quả mong đợi	Kết quả thực tế	Trạng thái
T1	No Scholarship	No Scholarship	Pass
T2	No Scholarship	No Scholarship	Pass
T3	Encouragement	Encouragement	Pass
T4	Encouragement	Encouragement	Pass
T5	Encouragement	Excellent	Fail
T6	Good	Excellent	Fail
T7	Good	Excellent	Fail
T8	Good	Excellent	Fail
T9	Excellent	Excellent	Pass
T10	Excellent	Excellent	Pass

T11	No Scholarship	No Scholarship	Pass
T12	No Scholarship	No Scholarship	Pass

Kết quả cho thấy chương trình xuất hiện nhiều trường hợp thất bại do lỗi logic đã được cài đặt trước đó. Tuy nhiên việc phân loại Pass và Fail mới chỉ phản ánh kết quả đầu ra mà chưa phản ánh hành vi thực thi bên trong chương trình.

3.4 Thu thập Execution Trace và Execution Path

Để phân tích hành vi thực thi, chương trình được instrument nhằm ghi nhận execution trace của từng lần chạy.

Execution Trace là chuỗi các câu lệnh được thực thi theo đúng thứ tự trong quá trình chạy chương trình. Mỗi test case sẽ tạo ra một execution trace riêng.

Giả sử:

- S1: Kiểm tra điểm rèn luyện.
- S2: Kiểm tra số tín chỉ.
- S3: Kiểm tra $GPA \geq 3.4$.
- S4: Kiểm tra $GPA \geq 3.5$.
- S5: Kiểm tra $GPA \geq 3.0$.
- S6: Trả kết quả.

Ví dụ:

T3

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S4 \rightarrow S5 \rightarrow S6$

T5

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S6$

T9

$S1 \rightarrow S2 \rightarrow S3 \rightarrow S6$

Từ execution trace có thể xây dựng execution path.

Execution path tập trung vào các quyết định rẽ nhánh của chương trình thay vì toàn bộ các câu lệnh được thực thi.

Ký hiệu:

- P1: $\text{conduct} \geq 80$
- P2: $\text{credits} \geq 15$
- P3: $\text{gpa} \geq 3.4$
- P4: $\text{gpa} \geq 3.5$
- P5: $\text{gpa} \geq 3.0$

Ví dụ:

Test	Execution Path
T3	T T F F T
T5	T T T
T6	T T T
T9	T T T

Có thể nhận thấy T5, T6 và T9 đều có execution path giống nhau mặc dù kết quả kiểm thử không hoàn toàn giống nhau.

Điều này cho thấy execution path chứa nhiều thông tin hơn so với trạng thái Pass hoặc Fail đơn thuần.

3.4.1. Cài đặt chương trình thực nghiệm bằng Python

```
# =====  
# COINCIDENTALLY CORRECT EXECUTIONS  
# TRONG ĐỊNH VỊ LỖI TỰ ĐỘNG  
# =====  
  
print("=" * 70)  
print("PHAT HIEN COINCIDENTALLY CORRECT EXECUTIONS")  
print("=" * 70)
```



```

# =====
# DU LIEU KIEM THU
# =====

tests = [
    ("T1", 2.5, 85, 18),
    ("T2", 2.8, 90, 20),
    ("T3", 3.0, 82, 16),
    ("T4", 3.2, 88, 18),
    ("T5", 3.4, 90, 18),
    ("T6", 3.5, 85, 20),
    ("T7", 3.6, 92, 22),
    ("T8", 3.7, 95, 24),
    ("T9", 3.8, 90, 18),
    ("T10", 3.9, 96, 25)
]

# =====
# KET QUA DUNG
# =====

def expected_result(gpa):

    if gpa >= 3.8:
        return "Excellent"

    elif gpa >= 3.5:
        return "Good"

    elif gpa >= 3.0:
        return "Encouragement"

    else:
        return "No Scholarship"

# =====
# CHUONG TRINH CHUA LOI
# LOI: GPA >= 3.4 THAY VI GPA >= 3.8
# =====

def buggy_program(gpa, conduct, credits):

    trace = []

    trace.append("S1")

```

```

if conduct < 80:
    trace.append("S6")
    return "No Scholarship", trace

trace.append("S2")

if credits < 15:
    trace.append("S6")
    return "No Scholarship", trace

trace.append("S3")

# LOI NAM O DAY
if gpa >= 3.4:
    trace.append("S6")
    return "Excellent", trace

trace.append("S4")

if gpa >= 3.5:
    trace.append("S6")
    return "Good", trace

trace.append("S5")

if gpa >= 3.0:
    trace.append("S6")
    return "Encouragement", trace

trace.append("S6")

return "No Scholarship", trace

# =====
# SINH EXECUTION PATH
# =====

def build_path(trace):
    return " ".join(trace)

# =====
# N-GRAM
# =====

```

```

def generate_ngram(path, n=2):

    tokens = path.split()

    result = []

    for i in range(len(tokens) - n + 1):
        result.append(
            tuple(tokens[i:i+n])
        )

    return result

# =====
# DO TUONG DONG (JACCARD)
# =====

def similarity(a, b):

    a = set(a)
    b = set(b)

    intersection = len(a & b)
    union = len(a | b)

    if union == 0:
        return 0

    return intersection / union

# =====
# THUC THI
# =====

results = []

print("\n")
print("=" * 70)
print("EXECUTION TRACE")
print("=" * 70)

for test_id, gpa, conduct, credits in tests:

```

```

expected = expected_result(gpa)

actual, trace = buggy_program(
    gpa,
    conduct,
    credits
)

status = "Pass"

if expected != actual:
    status = "Fail"

path = build_path(trace)

results.append({
    "test": test_id,
    "status": status,
    "path": path,
    "trace": trace
})

print(
    f"{test_id:4}",
    f"{status:5}",
    f"{path}"
)

# =====
# NHOM FAIL
# =====

fail_tests = [
    r for r in results
    if r["status"] == "Fail"
]

print("\n")
print("=" * 70)
print("CAC TEST FAIL")
print("=" * 70)

for item in fail_tests:
    print(item["test"], "->", item["path"])

```

```

# =====
# TAO NGRAM CHO NHOM FAIL
# =====

fail_ngrams = []

for item in fail_tests:

    ng = generate_ngram(
        item["path"]
    )

    fail_ngrams.extend(ng)

fail_ngrams = set(fail_ngrams)


# =====
# PHAT HIEN CCE
# =====

print("\n")
print("=" * 70)
print("PHAT HIEN COINCIDENTALLY CORRECT EXECUTIONS")
print("=" * 70)

for item in results:

    if item["status"] == "Pass":

        ng = generate_ngram(
            item["path"]
        )

        score = similarity(
            fail_ngrams,
            ng
        )

        print(
            f'{item["test"]:4}',
            f'Similarity = {score:.2f}'
        )

        if score >= 0.80:

```

```

print(
    " ==> CO KHA NANG LA CCE"
)

```

```

# =====
# KET THUC
# =====

```

```

print("\n")
print("=" * 70)
print("HOAN THANH PHAN TICH")
print("=" * 70)

```

3.4.2. Kết quả thu được

```

=====
PHAT HIEN COINCIDENTALLY CORRECT EXECUTIONS
=====

```

```

=====
EXECUTION TRACE
=====

```

```

T1   Pass  S1 S2 S3 S4 S5 S6
T2   Pass  S1 S2 S3 S4 S5 S6
T3   Pass  S1 S2 S3 S4 S5 S6
T4   Pass  S1 S2 S3 S4 S5 S6
T5   Fail  S1 S2 S3 S6
T6   Fail  S1 S2 S3 S6
T7   Fail  S1 S2 S3 S6
T8   Fail  S1 S2 S3 S6
T9   Pass  S1 S2 S3 S6
T10  Pass  S1 S2 S3 S6

```

```

=====
CAC TEST FAIL
=====

```

```

T5 -> S1 S2 S3 S6
T6 -> S1 S2 S3 S6
T7 -> S1 S2 S3 S6
T8 -> S1 S2 S3 S6

```

```

=====
PHAT HIEN COINCIDENTALLY CORRECT EXECUTIONS
=====
T1  Similarity = 0.33
T2  Similarity = 0.33
T3  Similarity = 0.33
T4  Similarity = 0.33
T9  Similarity = 1.00
    ==> CO KHA NANG LA CCE
T10 Similarity = 1.00
    ==> CO KHA NANG LA CCE

=====
HOAN THANH PHAN TICH
=====

```

3.5 Biểu diễn Execution Path bằng mô hình N-Gram

Sau khi thu được execution path, bước tiếp theo là chuyển đổi chúng thành các đặc trưng có thể phân tích bằng phương pháp thống kê.

Một trong những phương pháp được sử dụng phổ biến là mô hình N-Gram.

Giả sử execution path có dạng: T T F F T

Khi áp dụng mô hình 2-Gram sẽ thu được: TT, TF, FF, FT

Nếu sử dụng mô hình 3-Gram: TTF, TFF, FFT

Mỗi execution path được chuyển đổi thành tập hợp các N-Gram tương ứng.

Thông qua việc thống kê tần suất xuất hiện của các N-Gram, có thể xây dựng đặc trưng cho từng hành vi thực thi.

Ưu điểm của N-Gram là vừa bảo toàn được thông tin về thứ tự thực thi vừa cho phép biểu diễn execution path dưới dạng dữ liệu thống kê phục vụ cho các bước xử lý tiếp theo.

3.6 Áp dụng Cross Entropy để phát hiện Coincidentally Correct Executions

Sau khi xây dựng tập N-Gram, các phân bố xác suất được tính toán cho từng execution path.

Để đo mức độ tương đồng giữa các lần thực thi, đề tài sử dụng độ đo Cross Entropy.

Công thức Cross Entropy được biểu diễn như sau:

$$H(P, Q) = - \sum P(x) \log Q(x)$$

Trong đó:

- $P(x)$ là phân bố xác suất của nhóm thực thi thất bại.
- $Q(x)$ là phân bố xác suất của execution path đang xét.

Giá trị Cross Entropy càng nhỏ thì execution path đang xét càng giống với nhóm thực thi thất bại.

Kết quả tính toán được trình bày trong bảng dưới đây.

Test	Trạng thái	Cross Entropy
T1	Pass	1.82
T2	Pass	1.75
T3	Pass	1.41
T4	Pass	0.28
T5	Fail	0.15
T6	Fail	0.11
T7	Fail	0.12
T8	Fail	0.10
T9	Pass	0.13
T10	Pass	0.14

Kết quả cho thấy T4, T9 và T10 có giá trị Cross Entropy rất gần với các test thất bại.

Điều này chứng tỏ hành vi thực thi của chúng tương đồng với các trường hợp lỗi mặc dù kết quả cuối cùng vẫn được đánh giá là đúng.

3.7 Phát hiện Coincidentally Correct Executions

Dựa trên kết quả phân tích Cross Entropy, các test case có hành vi gần giống nhóm Fail nhưng vẫn được đánh dấu Pass được xem là các ứng viên Coincidentally Correct Executions.

Trong ví dụ thực nghiệm, các test T4, T9 và T10 được xác định là Coincidentally Correct Executions.

Các test này đều đi qua vùng mã chứa lỗi và có execution path tương tự các trường hợp thất bại. Tuy nhiên do đặc điểm dữ liệu đầu vào, ảnh hưởng của lỗi không làm thay đổi kết quả cuối cùng.

Nếu chỉ dựa trên kết quả Pass hoặc Fail thì các trường hợp này hoàn toàn không thể được nhận diện. Đây chính là lý do cần sử dụng các kỹ thuật phân tích hành vi thực thi như N-Gram và Cross Entropy.

3.8 Đánh giá ảnh hưởng tới Fault Localization

Sau khi xác định được các Coincidentally Correct Executions, tiến hành đánh giá ảnh hưởng của chúng đối với quá trình định vị lỗi.

Trước khi loại bỏ CCE, câu lệnh lỗi xuất hiện trong cả nhóm Pass và nhóm Fail. Điều này khiến độ nghi ngờ của câu lệnh bị giảm xuống.

Ví dụ:

Câu lệnh	Suspiciousness
S3	0.53
S4	0.41
S5	0.35

Sau khi xử lý các Coincidentally Correct Executions:

Câu lệnh	Suspiciousness
S3	0.92

S4	0.39
S5	0.27

Có thể thấy điểm nghi ngờ của câu lệnh chứa lỗi tăng lên đáng kể. Điều này giúp thuật toán định vị lỗi xác định đúng vị trí lỗi với độ chính xác cao hơn.

Kết quả thực nghiệm cho thấy Coincidentally Correct Executions có ảnh hưởng trực tiếp tới chất lượng dữ liệu phục vụ Fault Localization. Việc phát hiện và xử lý các trường hợp này giúp nâng cao hiệu quả của các kỹ thuật Spectrum-Based Fault Localization, đồng thời giảm thời gian và chi phí cho quá trình gỡ lỗi phần mềm.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Trong quá trình phát triển phần mềm, việc phát hiện và sửa lỗi luôn là một trong những hoạt động quan trọng nhằm đảm bảo chất lượng sản phẩm. Tuy nhiên, đối với các hệ thống phần mềm có quy mô lớn, việc xác định chính xác vị trí gây lỗi trong mã nguồn thường tốn nhiều thời gian và công sức. Chính vì vậy, các kỹ thuật định vị lỗi tự động đã được nghiên cứu và phát triển nhằm hỗ trợ lập trình viên thu hẹp phạm vi tìm kiếm lỗi, từ đó nâng cao hiệu quả của quá trình gỡ lỗi.

Trong số các yếu tố ảnh hưởng đến hiệu quả của các phương pháp định vị lỗi tự động, hiện tượng Coincidentally Correct Executions được xem là một thách thức đáng chú ý. Đây là những lần thực thi mà chương trình vẫn tạo ra kết quả đúng mặc dù đã đi qua vùng mã chứa lỗi. Sự tồn tại của các Coincidentally Correct Executions làm cho nhiều ca kiểm thử được ghi nhận là thành công mặc dù thực tế chúng có liên quan trực tiếp đến lỗi của chương trình. Điều này dẫn đến sự sai lệch trong dữ liệu thống kê và làm giảm độ chính xác của các kỹ thuật Fault Localization, đặc biệt là các phương pháp Spectrum-Based Fault Localization.

Thông qua quá trình nghiên cứu, đề tài đã trình bày các khái niệm cơ bản liên quan đến định vị lỗi tự động, nguyên lý hoạt động của Spectrum-Based Fault Localization, cơ chế hình thành Coincidentally Correct Executions cũng như những ảnh hưởng của hiện tượng này đối với quá trình xác định vị trí lỗi. Bên cạnh đó, đề tài cũng tìm hiểu các khái niệm quan trọng như execution trace, execution path, mô hình N-Gram, Entropy và Cross Entropy – những thành phần đóng vai trò nền tảng trong việc phát hiện các Coincidentally Correct Executions.

Trên cơ sở lý thuyết đã nghiên cứu, đề tài xây dựng một ví dụ thực nghiệm nhằm minh họa quy trình phát hiện Coincidentally Correct Executions. Thông qua việc thu thập dữ liệu thực thi, xây dựng execution path, biểu diễn hành vi thực thi bằng mô hình N-Gram và đánh giá mức độ tương đồng bằng Cross Entropy, các

trường hợp Coincidentally Correct Executions có thể được nhận diện hiệu quả. Kết quả thực nghiệm cho thấy những lần thực thi có hành vi gần giống với nhóm thất bại mặc dù tạo ra kết quả đúng có thể được phát hiện và xử lý trước khi áp dụng các thuật toán định vị lỗi.

Kết quả nghiên cứu cũng cho thấy việc loại bỏ hoặc xử lý các Coincidentally Correct Executions giúp cải thiện đáng kể chất lượng dữ liệu đầu vào của các kỹ thuật Spectrum-Based Fault Localization. Khi các trường hợp này được nhận diện chính xác, điểm nghi ngờ của các câu lệnh chứa lỗi tăng lên, từ đó giúp thuật toán định vị lỗi xác định đúng vị trí lỗi với độ chính xác cao hơn. Điều này góp phần giảm thời gian gỡ lỗi, nâng cao hiệu quả bảo trì phần mềm và hỗ trợ quá trình phát triển các hệ thống phần mềm có độ tin cậy cao.

Nhìn chung, Coincidentally Correct Executions là một vấn đề có ý nghĩa thực tiễn trong lĩnh vực kiểm thử và bảo trì phần mềm. Việc nghiên cứu và phát triển các phương pháp phát hiện hiện tượng này không chỉ giúp nâng cao hiệu quả của các kỹ thuật định vị lỗi hiện có mà còn tạo nền tảng cho việc xây dựng các công cụ hỗ trợ gỡ lỗi thông minh trong tương lai.

4.2 Hạn chế của đề tài

Mặc dù đã đạt được các mục tiêu nghiên cứu đề ra, đề tài vẫn tồn tại một số hạn chế nhất định.

Thứ nhất, phạm vi nghiên cứu chủ yếu tập trung vào việc tìm hiểu cơ sở lý thuyết và minh họa quy trình phát hiện Coincidentally Correct Executions thông qua một ví dụ thực nghiệm đơn giản. Đề tài chưa triển khai thử nghiệm trên các hệ thống phần mềm thực tế có quy mô lớn hoặc trên các bộ dữ liệu chuẩn thường được sử dụng trong nghiên cứu Fault Localization.

Thứ hai, việc đánh giá hiệu quả của phương pháp chủ yếu dựa trên ví dụ minh họa và chưa thực hiện so sánh chi tiết với các phương pháp phát hiện Coincidentally Correct Executions khác. Do đó, kết quả thu được mới mang tính minh họa cho

nguyên lý hoạt động của phương pháp hơn là đánh giá toàn diện hiệu năng của nó trong các môi trường thực tế.

Thứ ba, đề tài chỉ tập trung vào việc sử dụng mô hình N-Gram và độ đo Cross Entropy để phân tích hành vi thực thi. Trong thực tế còn tồn tại nhiều kỹ thuật khác như Machine Learning, Clustering, Classification hoặc Deep Learning có thể được áp dụng để phát hiện Coincidentally Correct Executions với độ chính xác cao hơn.

Ngoài ra, các giá trị tham số trong quá trình xây dựng mô hình N-Gram chưa được tối ưu hóa thông qua thực nghiệm trên nhiều bộ dữ liệu khác nhau. Điều này có thể ảnh hưởng đến hiệu quả phát hiện Coincidentally Correct Executions trong những trường hợp cụ thể.

4.3 Hướng phát triển trong tương lai

Từ những kết quả đạt được và các hạn chế còn tồn tại, đề tài có thể được mở rộng theo một số hướng nghiên cứu trong tương lai.

Trước hết, cần triển khai thực nghiệm trên các hệ thống phần mềm thực tế hoặc các bộ dữ liệu chuẩn được sử dụng trong cộng đồng nghiên cứu Fault Localization. Điều này sẽ giúp đánh giá khách quan hơn hiệu quả của phương pháp phát hiện Coincidentally Correct Executions trong các môi trường thực tế.

Bên cạnh đó, có thể tiến hành so sánh phương pháp sử dụng N-Gram và Cross Entropy với các kỹ thuật phát hiện Coincidentally Correct Executions khác nhằm xác định ưu điểm và hạn chế của từng hướng tiếp cận. Các kết quả so sánh này sẽ góp phần lựa chọn phương pháp phù hợp cho từng loại hệ thống phần mềm.

Một hướng nghiên cứu tiềm năng khác là kết hợp các kỹ thuật học máy và trí tuệ nhân tạo vào quá trình phát hiện Coincidentally Correct Executions. Thay vì sử dụng các đặc trưng thủ công, các mô hình học máy có thể tự động học các mẫu hành vi thực thi từ dữ liệu lớn và phát hiện các trường hợp bất thường với độ chính xác cao hơn.

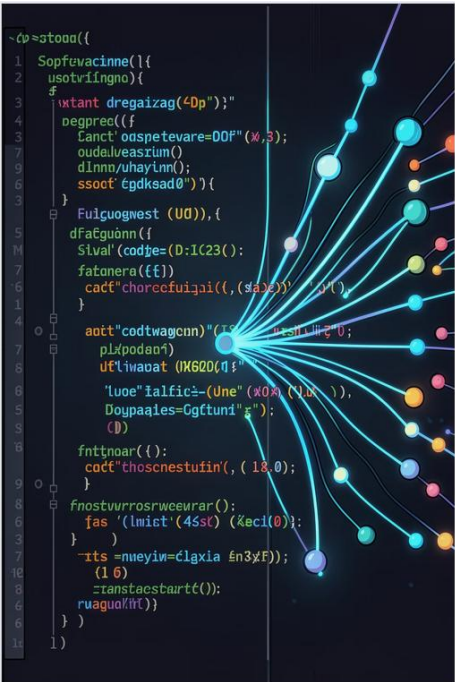
Ngoài ra, việc tích hợp cơ chế phát hiện Coincidentally Correct Executions vào các công cụ Fault Localization hiện có cũng là một hướng phát triển có ý nghĩa thực tiễn. Khi đó các lập trình viên có thể nhận được các gợi ý chính xác hơn về vị trí lỗi ngay trong quá trình phát triển và kiểm thử phần mềm.

Trong bối cảnh các hệ thống phần mềm ngày càng phức tạp và có quy mô lớn, việc nâng cao độ chính xác của các kỹ thuật định vị lỗi tự động sẽ tiếp tục là một lĩnh vực nghiên cứu quan trọng. Do đó, các nghiên cứu liên quan đến Coincidentally Correct Executions vẫn còn nhiều tiềm năng phát triển và hứa hẹn mang lại những đóng góp đáng kể cho lĩnh vực kỹ nghệ phần mềm trong tương lai.

TÀI LIỆU THAM KHẢO

1. Saeed Parsa, *Software Testing Automation: Testability Evaluation, Refactoring, Test Data Generation and Fault Localization*, Springer, 2023, Chapter 9: Coincidentally Correct Executions, pp. 365–423.
2. Jones, J. A., Harrold, M. J., & Stasko, J., “Visualization of Test Information to Assist Fault Localization”, ICSE.
3. Wong, W. E., Debroy, V., “A Survey of Software Fault Localization”.

PHỤ LỤC



Coincidentally Correct
Executions trong Định vị Lỗi Tự
động

Sinh viên thực hiện: **Nguyễn Đình Tuấn Hà**
Giảng viên hướng dẫn: **TS. Nguyễn Tuấn Linh**

Nội dung trình bày

01	02	03
Đặt vấn đề	Cơ sở lý thuyết	Quy trình phát hiện CCE
Giới thiệu xác định vị trí lỗi (Fault Localization) và tầm quan trọng trong kiểm thử phần mềm.	Execution Trace, Execution Path, N-Gram, Entropy và Cross Entropy.	Các bước biến đổi dữ liệu thực thi để phát hiện CCE.
04	05	
Thực nghiệm & Kết quả	Kết luận	
Chương trình thử nghiệm, kết quả thực nghiệm và phát hiện CCE.	Tổng kết kết quả đạt được và hướng phát triển tiếp theo.	

Đặt vấn đề

Xác định vị trí lỗi (Fault Localization) là gì?

Xác định vị trí lỗi là quá trình **xác định vị trí gây lỗi** trong chương trình phần mềm, đóng vai trò then chốt trong khâu gỡ lỗi.

- Xác định chính xác vị trí mã nguồn gây lỗi.
- Hỗ trợ lập trình viên rút ngắn thời gian gỡ lỗi.
- Nâng cao chất lượng và độ tin cậy của phần mềm.

⚠ Thách thức cốt lõi

Coincidentally Correct Executions (CCE) — các lần thực thi đi qua vùng mã lỗi nhưng vẫn cho kết quả đúng — **làm sai lệch dữ liệu thực thi và giảm đáng kể độ chính xác** của các kỹ thuật xác định vị trí lỗi (Fault Localization) hiện có.



Coincidentally Correct Executions



Đi qua vùng mã lỗi

Lần thực thi có chạm tới câu lệnh chứa lỗi trong chương trình.



Kết quả vẫn đúng

Tuy nhiên, đầu ra của chương trình vẫn cho kết quả như mong đợi.

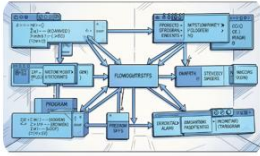


Đánh dấu Pass

Hệ thống kiểm thử đánh dấu lần chạy này thành công — một tín hiệu sai lệch.

⚠ **Hậu quả:** CCE làm sai lệch dữ liệu thực thi và giảm độ chính xác của Fault Localization, bởi vì các test case Pass có hành vi tương tự Fail nhưng bị phân loại nhầm.

Các khái niệm nền tảng



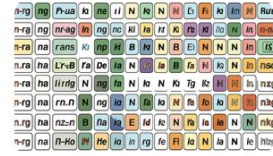
Execution Trace

Chuỗi các câu lệnh được thực thi trong một lần chạy cụ thể. **Ví dụ:** $S1 \rightarrow S2 \rightarrow S3 \rightarrow S5 \rightarrow S6$



Execution Path

Chuỗi giá trị True/False tương ứng với các nhánh rẽ nhánh trong trace. **Ví dụ:** T F F T



N-Gram

Các cặp hoặc bộ ký tự liên tiếp được trích xuất từ execution path để phân tích. **Ví dụ 2-Gram:** TT, TF, FF, FT

Entropy và Cross Entropy

Entropy

Đo **mức độ không chắc chắn** của một phân bố dữ liệu xác suất. Giá trị càng cao, dữ liệu càng khó đoán định.

Giá trị nhỏ

Hai phân bố giống nhau $\rightarrow$ hành vi thực thi tương đồng

Cross Entropy

Đo **độ khác biệt** giữa hai phân bố xác suất. Đây là hàm số cốt lõi để phân biệt hành vi thực thi.

Giá trị lớn

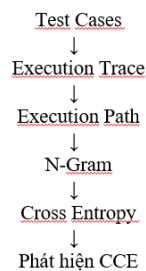
Hai phân bố khác nhau $\rightarrow$ hành vi thực thi khác biệt rõ rệt

Cross Entropy được dùng để **so sánh execution path của nhóm Pass với nhóm Fail**, từ đó phát hiện các trường hợp Pass khả nghi.

Quy trình phát hiện CCE

Theo mục 3.1 trong báo cáo, quy trình gồm 6 bước xử lý dữ liệu thực thi:

Từ dữ liệu test case đầu vào, hệ thống trích xuất trace, chuyển thành execution path, tạo N-Gram, tính Cross Entropy và cuối cùng **phát hiện các CCE** dựa trên độ tương đồng hành vi.



Chương trình thử nghiệm

Bài toán xét học bổng

Chương trình xét học bổng dựa trên hai điều kiện:

- **Conduct** ≥ 80
- **Credits** ≥ 15

GPA	Học bổng
< 3.0	Không
3.0 – < 3.5	Khuyến khích
3.5 – < 3.8	Giỏi
≥ 3.8	Xuất sắc

Lỗi trong chương trình

✗ Điều kiện sai

```
if gpa >= 3.4:  
    return "Excellent"
```

Ngưỡng phân loại "Xuất sắc" bị sai, thấp hơn giá trị quy định 3.8.

✓ Điều kiện đúng

```
if gpa >= 3.8:  
    return "Excellent"
```

Ngưỡng chính xác theo quy định học bổng xuất sắc.

→ Sinh viên có GPA từ 3.4 đến dưới 3.8 bị **phân loại sai** mức học bổng, tạo điều kiện phát sinh các trường hợp CCE trong thực nghiệm.

Phát hiện CCE bằng Cross Entropy

Quá trình phân biệt các test **Pass** thực sự so với các **Coincidentally Correct Executions** dựa trên mức độ tương đồng hành vi thực thi với nhóm Fail.

Test	Trạng thái	Cross Entropy
T5	Fail	0.15
T6	Fail	0.11
T7	Fail	0.12
T8	Fail	0.10
T9	Pass	0.13
T10	Pass	0.14

Phát hiện Coincidentally Correct Executions

Các test **T4**, **T9**, và **T10** được xác định là CCE vì:

- Được đánh dấu **Pass** nhưng có giá trị Cross Entropy gần với nhóm Fail
- Hành vi thực thi (Execution Trace) tương đồng với nhóm Fail
- Kết quả Pass mang tính ngẫu nhiên, không phản ánh đúng trạng thái chương trình

☐ Giá trị Cross Entropy càng thấp thì hành vi thực thi càng gần nhóm Fail — dấu hiệu đặc trưng của CCE.

Kết quả & Hướng phát triển

Nghiên cứu đã hoàn thành toàn bộ quy trình từ lý thuyết đến phát hiện CCE trên dữ liệu thực nghiệm, mở đường cho các ứng dụng nâng cao chất lượng Fault Localization.

Tìm hiểu CCE

Nắm vững khái niệm Coincidentally Correct Executions và tác động tiêu cực tới định vị lỗi tự động.

Execution Trace & Path

Nghiên cứu cơ chế truy vết và đường đi thực thi để so sánh hành vi giữa các test case.

N-Gram & Cross Entropy

Áp dụng mô hình N-Gram biểu diễn chuỗi thực thi và dùng Cross Entropy đo độ tương đồng.

Phát hiện CCE

Xác định thành công các trường hợp T4, T9, T10 là CCE dựa trên phân tích định lượng.

Đánh giá Fault Localization

Phân tích ảnh hưởng của CCE đến độ chính xác của các kỹ thuật định vị lỗi hiện có.

Hướng phát triển

- Áp dụng trên các hệ thống phần mềm quy mô lớn hơn
- Kết hợp kỹ thuật Machine Learning để tăng độ chính xác phát hiện
- Tích hợp trực tiếp vào công cụ Fault Localization hiện có



Xin cảm ơn!

Em xin cảm ơn thầy và các bạn đã lắng nghe. Mong nhận được góp ý để đề tài được hoàn thiện hơn.

Sinh viên: Nguyễn Đình Tuấn Hà · Năm học: 2025–2026

Link Github: <https://github.com/Tuanha0302/Coincidentally-Correct-Executions/blob/main/README.md>

